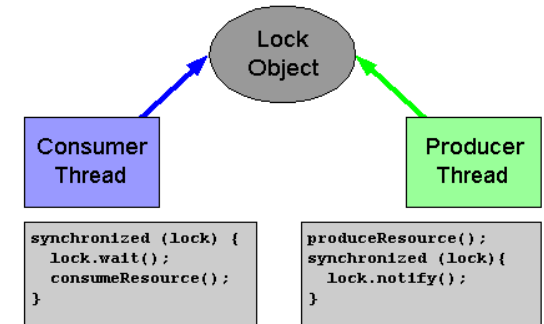




Høgskulen
på Vestlandet

DAT108 - Programmering og webapplikasjoner

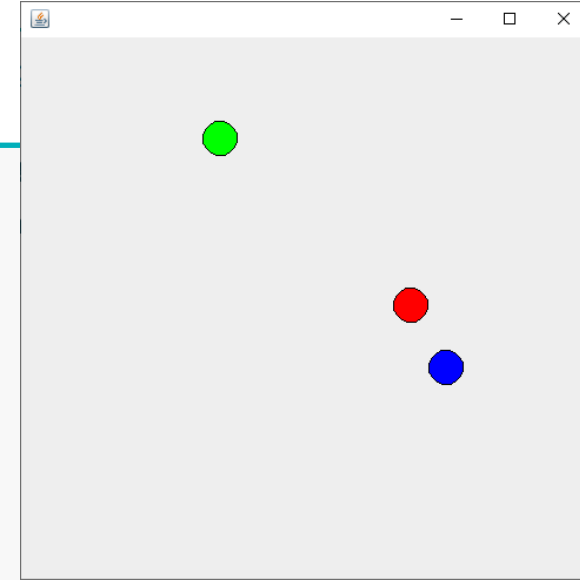
F08 – Baller, problemstillinger og Locks



Lars-Petter Helland
Bergen
11. september 2026

Plan for i dag

- › Kjapp recap av synchronized, wait, notify, notifyAll
- › En liten animasjonsoppgave :)
- › --
- › Noen vanlige problemstillinger når vi låser tråder ute
- › Låser (Locks) i Java
- › Lock, ReentrantLock, Condition (som alternativ til sync, wait, notify)
- › ... m/ flere Java-eksempler



Læringsressurser til i dag

Fra Java Concurrency and Multithreading Tutorial (<http://tutorials.jenkov.com/java-concurrency/index.html>):

25. Locks in Java

Fra Java Lock and Condition Example using Producer Consumer Solution (<https://javarevisited.blogspot.com/2015/06/java-lock-and-condition-example-producer-consumer.html>):

`java.util.concurrent.locks.Lock` (interface)

`java.util.concurrent.locks.ReentrantLock` (klasse)

`java.util.concurrent.Condition` (interface)



Recap: synchronized, wait, notify, notifyAll

- › Deler av koden som vi ikke kan tillate at flere tråder jobber mot samtidig kalles en kritisk seksjon.
- › Vi kan merke blokker eller metoder med **synchronized**. Dette sikrer at kun én tråd om gangen kan utføre koden i blokker merket på denne måten.
- › **synchronized** bruker et objekt som lås (monitor, ball, ...). Det er kun den tråden som holder låsen som får utføre kode i de "synkroniserte" blokkene for akkurat denne låsen.
- › En **wait**-melding på låsen setter gjeldende tråd "på vent" slik at en annen tråd kan få låsen og gå inn i synchronized-blokk.
- › En **notify**-melding på låsen sier at "ventende" tråder kan være klar til å fortsette utføring av kode (etter å ha fått låsen igjen) der de ga fra seg låsen.



Det kan f.eks. se slik ut i Java

```
public class Main {  
  
    private static String melding;  
  
    public static void main(String[] args) {  
  
        Object lock = new Object();
```

Felles monitor / lås

Tråd2

```
@Override  
public void run() {  
    synchronized (lock) {  
        melding = "Dette er en melding!";  
        lock.notifyAll();  
    }  
}
```

Tråd1

```
@Override  
public void run() {  
    synchronized (lock) {  
        while (melding == null) {  
            try {  
                lock.wait();  
            } catch (InterruptedException e) {  
            }  
        }  
        System.out.println(melding);  
    }  
}
```



Evt. slik (nesten samme eksempel)

```
public class Melding {  
  
    private String innhold;  
  
    public synchronized String getInnhold() {  
        while (!harInnhold()) {  
            try {  
                wait();  
            } catch (InterruptedException e) {  
            }  
        }  
        return innhold;  
    }  
  
    public synchronized void setInnhold(String innhold) {  
        this.innhold = innhold;  
        notifyAll();  
    }  
}
```

Her har vi flyttet
"synkroniseringen" inn i den
delte ressursen slik at den blir
trådsikker. Melding vil automatisk
blokkere tråder etter behov.

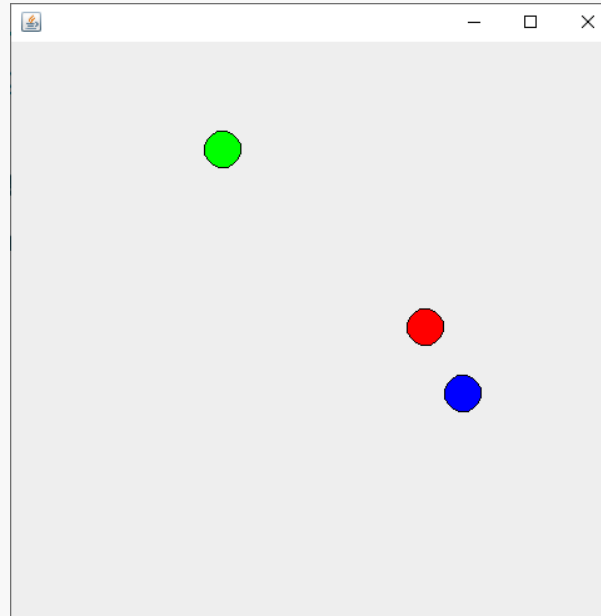


Eksempel/oppgave - baller

GitHub: <https://github.com/lars-petter-helland/dat108-h2026>

Prosjekt: **f08-baller-reentrantlock**

--- **baller.***

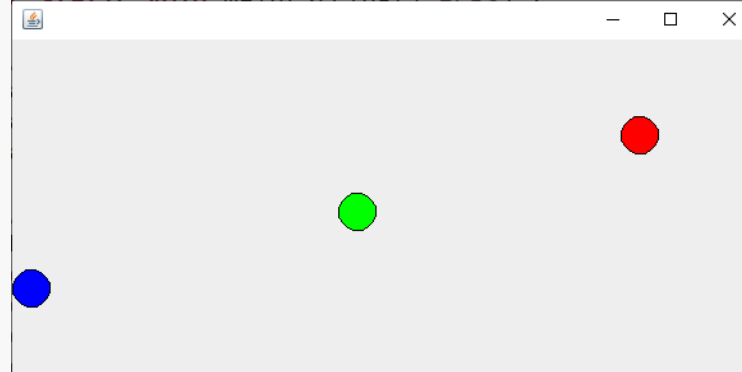


Eksempel/oppgave - ballstafett

GitHub: <https://github.com/lars-petter-helland/dat108-h2026>

Prosjekt: **f08-baller-reentrantlock**

--- **ballstafett.***







Vi skal se på noen problemstillinger med tråder

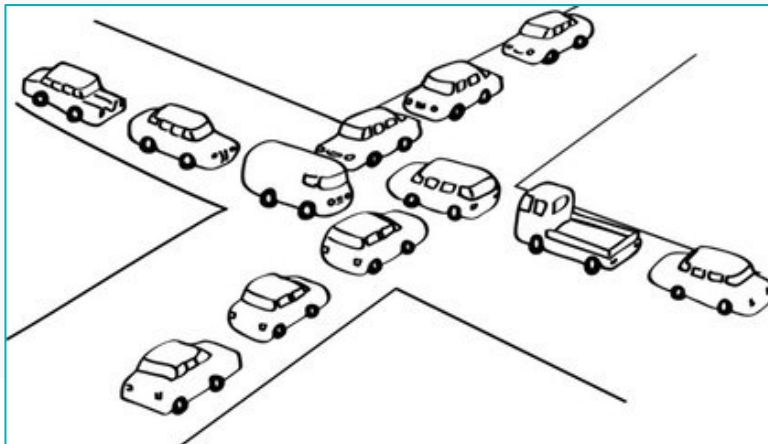
- › Det finnes mange problemstillinger ved låsing og "synkronisering", og vi skal se på et par. Dere må kjenne og forstå disse begrepene:
 - › **Vranglås** (Deadlock ... og på gøy: Livelock)
 - › **Utsulting** og **rettferdighet** (Starvation og fairness)
 - › **Nøstet monitor-utelåsing** (Nested monitor lockout)
 - › **Betingelse som glipper** (Slipped condition)
- › Kapitlene 19 - 24 omtaler disse tingene, men går av og til litt for langt til at vi bruker det som pensum. Les dem gjerne likevel!



Vranglås (deadlock)

- › En vranglås er en situasjon der en tråd1 venter på lås holdt av tråd2, mens tråd2 venter på lås holdt av tråd1.
- › Dette kan f.eks. oppstå hvis samarbeidende tråder tillates å "ta" låser i ulik rekkefølge.
- › Et vanlig "enkelt" tiltak er å sørge for at låser alltid "tas" i samme rekkefølge, f.eks. at lås A må tas før lås B ...

```
public void run1() {  
    synchronized(lockA) {  
        //...  
        synchronized(lockB) {  
            //...  
        }  
    }  
}  
  
public void run2() {  
    synchronized(lockB) {  
        //...  
        synchronized(lockA) {  
            //...  
        }  
    }  
}
```



Vranglås (livelock)

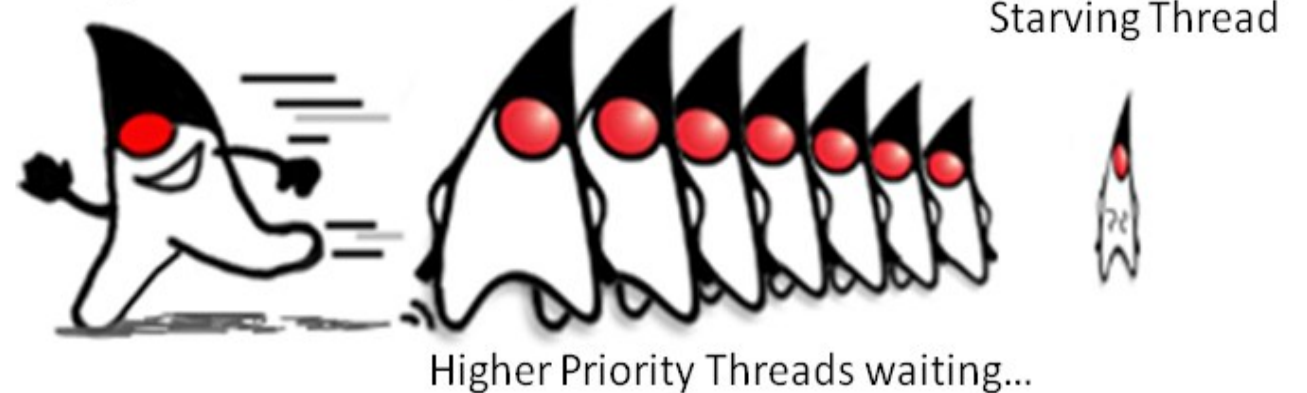
- › En livelock er en situasjon der man ikke er helt låst, men der begge trådene bruker samme strategi og hele tiden ender opp i en låst tilstand.



Utsulting og rettferdighet

- › Man kan ha situasjoner der noen tråder er mer "ivrige" enn andre, og det er også mulig å gi tråder ulik prioritet.
- › Dette kan føre til at de samme trådene hele tiden får CPU-en, mens andre tråder alltid blir stående å vente (sulte).
- › Vi kan sikre rettferdighet ved at man tar hensyn til om tråder har ventet lenge ++.
- › For vanlig Java-utvikling er ikke dette noe å tenke på.

Running Java Thread



Betingelse som glipper (Jenkov)

```
public class Lock {  
    private boolean isLocked = true;  
  
    public void lock(){  
        synchronized(this){  
            while(isLocked){  
                try{  
                    this.wait();  
                } catch (InterruptedException e){  
                    //do nothing, keep waiting  
                }  
            }  
        }  
  
        synchronized(this){  
            isLocked = true;  
        }  
    }  
  
    public synchronized void unlock(){  
        isLocked = false;  
        this.notify();  
    }  
}
```

Her har vi gitt fra oss låsen et lite øyeblikk. Kan vi fremdeles anta at isLocked er false?

```
public class Lock {  
    private boolean isLocked = true;  
  
    public void lock(){  
        synchronized(this){  
            while(isLocked){  
                try{  
                    this.wait();  
                } catch (InterruptedException e){  
                    //do nothing, keep waiting  
                }  
            }  
            isLocked = true;  
        }  
    }  
  
    public synchronized void unlock(){  
        isLocked = false;  
        this.notify();  
    }  
}
```

Husker dere feil plassering av System.out.println forrige gang?



Locks in Java

- › Skal nå se litt på direkte bruk av låser (i Java). Altså ikke via `synchronized`.
- › En lås (lock) er en mekanisme for trådsynkronisering på linje med bruken av `synchronized`, men er litt mer fleksibel og kraftig.
- › I Java finnes et bibliotek med klasser for det vi skal se på nå (i pakken `java.util.concurrent.locks`), men vi skal først se litt på hvordan låser kan implementeres ved å bruke `synchronized`, `wait` og `notify`.
- › Vi skal se på en enkel lås (**Lock**)
- › Vi skal se hva re-entrant betyr (**ReentrantLock**)
- › Vi skal se hvordan vi kan sende signaler (**Condition**)



Først et lite tenkt eksempel (Jenkov)

Kunne vært skrevet slik i stedet

```
public class Counter{  
    private int count = 0;  
  
    public int inc(){  
        synchronized(this){  
            return ++count;  
        }  
    }  
}
```

```
public class Counter{  
  
    private Lock lock = new Lock();  
    private int count = 0;  
  
    public int inc(){  
        lock.lock();  
        int newCount = ++count;  
        lock.unlock();  
        return newCount;  
    }  
}
```

En mulig Lock-implementasjon

```
public class Lock{  
    private boolean isLocked = false;  
  
    public synchronized void lock()  
    throws InterruptedException{  
        while(isLocked){  
            wait();  
        }  
        isLocked = true;  
    }  
  
    public synchronized void unlock(){  
        isLocked = false;  
        notify();  
    }  
}
```

Lock-klassen kunne kanskje
vært implementert slik

Reentrancy

```
public class Reentrant2{  
    Lock lock = new Lock();  
  
    public outer(){  
        lock.lock();  
        inner();  
        lock.unlock();  
    }  
  
    public synchronized inner(){  
        lock.lock();  
        //do something  
        lock.unlock();  
    }  
}
```

Ville Lock-implementasjonen
vår fungert greit her?

Reentrancy

```
public class Reentrant2{  
  
    Lock lock = new Lock();  
  
    public outer(){  
        lock.lock();  
        inner();  
        lock.unlock();  
    }  
  
    public synchronized inner(){  
        lock.lock();  
        //do something  
        lock.unlock();  
    }  
}
```

Nei. Problemet er at vi aldri får gjort `lock.lock()` i `inner()`. Låsen er allerede låst.

Reentrant lock

```
public class Lock{

    boolean isLocked = false;
    Thread   lockedBy = null;
    int      lockedCount = 0;

    public synchronized void lock()
    throws InterruptedException{
        Thread callingThread = Thread.currentThread();
        while(isLocked && lockedBy != callingThread){
            wait();
        }
        isLocked = true;
        lockedCount++;
        lockedBy = callingThread;
    }

    public synchronized void unlock(){
        if(Thread.curentThread() == this.lockedBy){
            lockedCount--;

            if(lockedCount == 0){
                isLocked = false;
                notify();
            }
        }
    }
}
```

En mulig løsning der vi tillater en tråd å re-entre kritisk seksjon hvis den allerede holder låsen!

Nå: Skal "låse" ved bruk av Java sine Lock og Condition i stedet for synchronized, wait og notifyAll ...

```
public class Melding {  
  
    private String innhold;  
  
    public synchronized String getInnhold() {  
        while (!harInnhold()) {  
            try {  
                wait();  
            } catch (InterruptedException e) {  
            }  
        }  
        return innhold;  
    }  
  
    public synchronized void setInnhold(String innhold) {  
        this.innhold = innhold;  
        notifyAll();  
    }  
}
```



Eksempel - Lock, ReentrantLock og Condition

GitHub: <https://github.com/lars-petter-helland/dat108-h2026>

Prosjekt: **f08-baller-reentrantlock**

--- **lockawaitsignal.***



Oppsummert fra eksempelet

```
private Lock laasen = new ReentrantLock();  
private Condition innholdSatt = laasen.newCondition();
```

```
public void lockIdiom() {  
    laasen.lock();  
    try {  
        //gjør greiene  
    } finally {  
        laasen.unlock();  
    }  
}
```

```
innholdSatt.await();
```

```
innholdSatt.signalAll();
```

- › Et par fordeler med Lock fremfor synchronized:
 - › Kan ha flere ulike Conditions, dvs. gi signal kun til relevante tråder
 - › Kan sette timeout
 - › Har i tillegg en del nyttige metoder i Lock- og Condition-APIet

Videre

Neste gang mandag:

- › Hvordan lage trådsikker kode, inkl. Collections
- › ConcurrentList, AtomicInteger, ...
- › Callable / Future / CompletableFuture
- › Ikke-blokkerende / asynkron / reaktiv programmering

Øving ...:

- › Jobbe med oppgaver! inkl. Oblig 2. **Innleveringsfrist søndag 20.09.**

Neste uke fredag:

- › Start modul **JavaScript** med lærer **Bjarte Kileng**

